

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze

9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Comparative Analysis

1. Sequential File Organization vs Heap File Organization

Question: Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Answer

Sequential and heap file organizations differ mainly in how records are stored and how efficiently they can be accessed.

Feature	Sequential File Organization	Heap File Organization
Record arrangement	Records are stored in a specific order, usually based on a key	Records are stored in the order in which they arrive
Insertion	Slower because the correct position may need to be found	Very fast; normally added at the end or in a free space
Deletion	More costly because the sequence may need maintenance	Relatively simple; record can be marked/deleted
Exact search	Efficient when searching by the ordering key	Slow because records may need to be scanned
Sequential retrieval	Very efficient	Efficient, but records are not ordered
Range queries	Highly efficient	Less efficient
Maintenance	Requires maintaining the ordering	Simple maintenance
Best suited for	Reports, ordered records and range processing	Frequently inserted records

Example

Suppose student records are stored according to Student_ID.

Sequential file:

101 → 105 → 110 → 115 → 120

If we want students from ID 105 to 115, the records can be accessed efficiently.

Heap file:

110 → 101 → 120 → 105 → 115

Records are not maintained in a particular order.

Analysis

For **frequent insertions**, heap organization is better because a new record can usually be added without rearranging existing records.

For **ordered retrieval and range queries**, sequential organization is better because records are already arranged according to the key.

Conclusion

Heap organization is preferable for write-heavy workloads, while sequential organization is preferable when ordered retrieval and range queries are important.

2. Clustered Indexing vs Non-Clustered Indexing

Question: Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

Answer

Clustered and non-clustered indexes differ in how the index relates to the physical order of the records.

Feature	Clustered Index	Non-Clustered Index
Data order	Data records are physically organized according to the index key	Physical order of records is independent of the index
Number per table	Usually one	Multiple can be created
Storage	Closely associated with data storage	Requires separate index structure
Range queries	Very efficient	Efficient, but may require additional record lookups
Exact search	Fast	Fast
Maintenance	Can be costly when physical order changes	Index must be updated when records change
Example	Employee records ordered by Employee_ID	Index on Employee_Name

Example

Consider:

EMPLOYEE

Employee_ID | Name | Department

101 | Arun | CSE

102 | Ravi | ECE

103 | Kiran | CSE

If the table is physically organized according to Employee_ID, a clustered index on Employee_ID follows that physical arrangement.

A non-clustered index on Name may look conceptually like:

Arun → Record location

Kiran → Record location

Ravi → Record location

The index stores the search key and a reference to the actual record.

Analysis

A clustered index is particularly useful for **range queries**, because related records are physically close together.

A non-clustered index is useful when users need to search the same table using **multiple different attributes**.

Conclusion

Use a **clustered index** when the physical ordering of records should match an important access pattern, and use **non-clustered indexes** for additional search attributes.

3. Primary Indexing vs Secondary Indexing

Question: Compare primary indexing and secondary indexing based on storage requirements and search performance.

Answer

Primary and secondary indexes provide different methods of accessing records.

Feature	Primary Index	Secondary Index
Based on	Primary/ordering key	Non-ordering search attribute
Data organization	Usually associated with sorted data	Data need not be physically sorted on the secondary key
Number	Generally one primary ordering index	Multiple secondary indexes can exist
Storage	Usually lower, especially when sparse	Can require more storage
Search performance	Very efficient for primary-key access	Efficient for searches on secondary attributes
Range queries	Efficient when based on ordering key	Depends on secondary index structure
Maintenance	Relatively simpler	More maintenance when many indexes exist

Example

Consider:

STUDENT

Student_ID | Name | Department

A primary index can be created on:

Student_ID

A secondary index can be created on:

Department

For example:

CSE → locations of CSE students

ECE → locations of ECE students

AIML → locations of AIML students

Storage Analysis

A primary index can often be implemented as a **sparse index**, where only selected search-key values are stored.

A secondary index is commonly **dense**, because records are not necessarily physically ordered according to the secondary attribute.

Therefore, secondary indexes may require more index entries and more storage.

Search Analysis

If the query is:

```
SELECT * FROM Student
```

```
WHERE Student_ID = 105;
```

the primary index is highly suitable.

For:

```
SELECT * FROM Student
```

```
WHERE Department = 'CSE';
```

the secondary index is useful.

Conclusion

Primary indexing generally requires less storage and provides efficient access through the primary ordering key, while secondary indexing provides flexible access through additional attributes at the cost of extra storage and maintenance.

4. Static Hashing vs Dynamic Hashing

Question: Analyze the differences between static hashing and dynamic hashing in database systems.

Answer

Hashing maps a search key to a bucket using a hash function. Static and dynamic hashing differ mainly in how they handle database growth.

Feature	Static Hashing	Dynamic Hashing
Number of buckets	Fixed	Can change dynamically
Database growth	Less flexible	Highly flexible
Overflow	Usually handled using overflow buckets/chaining	Handled through bucket splitting
Performance with growth	Can degrade due to overflow	Remains more stable
Implementation	Simple	More complex
Storage management	Fixed initially	Adjusts according to data
Suitable for	Relatively stable data	Frequently growing databases

Static Hashing

Suppose there are 10 buckets:

$$h(\text{key}) = \text{key} \bmod 10$$

The number of buckets remains fixed.

If many records map to the same bucket:

Bucket 5 → Record A → Record B → Record C → Overflow

the overflow chain can become long.

Dynamic Hashing

Dynamic hashing can create or split buckets as records increase.

For example:

Before:

Bucket 0

Bucket 1

After overflow:

Bucket 0

Bucket 1

Bucket 2

Bucket 3

The structure expands according to the workload.

Analysis

Static hashing is simpler and can be efficient when the number of records is predictable.

Dynamic hashing is better when the database size changes frequently because it reduces the performance problems caused by long overflow chains.

Conclusion

Static hashing is suitable for stable databases, whereas dynamic hashing is more suitable for growing and changing databases. For modern applications with unpredictable growth, dynamic hashing generally provides better scalability.

5. B-Tree vs B+ Tree

Question: Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Answer

B-Tree and B+ Tree are balanced tree structures used for efficient database indexing.

Feature	B-Tree	B+ Tree
Data storage	Keys and records may appear in internal and leaf nodes	Actual records/data pointers are stored at leaf level
Internal nodes	Can contain data	Mainly contain keys and child pointers
Leaf nodes	May not be linked	Leaf nodes are linked
Exact search	Efficient	Efficient
Range search	Good	Excellent
Sequential access	Less convenient	Very efficient
Fan-out	Lower because internal nodes may store data	Higher because internal nodes mainly store keys
Database usage	Less commonly preferred	Widely preferred for database indexing

B-Tree Structure

[20]

/ \

[5,10] [30,40]

Keys and associated data can exist at different levels.

B+ Tree Structure

[20]

/ \

[5,10] → [20,30,40]

The leaf nodes are linked.

Why B+ Tree is better for large databases

Suppose a database needs:

Find all records with keys from 1000 to 5000

In a B+ Tree, once the first matching leaf is located, the database can follow the linked leaf nodes:

[1000,1100] → [1200,1300] → [1400,1500] → ...

This makes range retrieval highly efficient.

Analysis

B+ Trees also have a high **fan-out**, meaning more keys can fit in each internal node. This reduces the height of the tree and consequently reduces the number of disk accesses.

Conclusion

For **large-scale database applications**, **B+ Tree is generally preferred** because it provides high fan-out, efficient disk access, excellent range queries, and linked leaf nodes.

6. Linear Hashing vs Extendible Hashing

Question: Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Answer

Both linear hashing and extendible hashing are dynamic hashing techniques, but they expand the hash structure differently.

Feature	Linear Hashing	Extendible Hashing
Directory	Does not require a directory	Uses a directory
Expansion	Buckets split gradually	Bucket split can cause directory expansion
Bucket management	Controlled using a split pointer	Controlled using global and local depth
Directory doubling	Not required	May occur
Scalability	Good	Very good
Memory overhead	Lower	Higher because of directory
Implementation	Relatively simple	More complex
Overflow handling	Gradual bucket splitting	Bucket splitting and directory management

Linear Hashing

Linear hashing uses a **split pointer**.

Conceptually:

Bucket 0

Bucket 1

Bucket 2 ← split next

Bucket 3

Buckets are split gradually rather than expanding the entire structure at once.

Extendible Hashing

Extendible hashing uses a directory:

Directory

|

00 → Bucket

01 → Bucket

10 → Bucket

11 → Bucket

When overflow occurs, a bucket can be split. If the local depth reaches the global depth, the directory can double.

Analysis

Linear hashing has lower directory overhead and expands gradually.

Extendible hashing can provide very direct bucket selection and fast access, but the directory consumes additional memory and can grow.

Conclusion

Linear hashing is attractive when gradual growth and lower directory overhead are important. Extendible hashing is preferable when very fast bucket identification and controlled dynamic expansion are required.

7. Dense Indexing vs Sparse Indexing

Question: Compare dense indexing and sparse indexing with respect to memory usage and access speed.

Answer

Dense and sparse indexes differ mainly in the number of index entries they contain.

Feature	Dense Index	Sparse Index
Index entries	Usually one for every record/search key	Only selected records/blocks
Memory usage	Higher	Lower
Search speed	Very fast	Slightly slower
Storage overhead	High	Low
Data organization	Can work without the same level of ordering requirement	Usually requires sorted/ordered data
Maintenance	More entries must be updated	Fewer entries need updating
Suitable for	Fast direct access	Large datasets where memory efficiency is important

Dense Index Example

Suppose:

101

102

103

104

105

A dense index contains:

101 → Record

102 → Record

103 → Record

104 → Record

105 → Record

Every record has an index entry.

Sparse Index Example

A sparse index may contain:

101 → Block 1

104 → Block 2

The database first finds the appropriate block and then searches within that block.

Analysis

Dense indexing provides faster access because the exact record can be identified directly from the index.

However, it consumes more memory.

Sparse indexing requires less memory but may require an additional search within the selected data block.

Conclusion

Dense indexing should be preferred when search speed is the highest priority and sufficient memory is available. Sparse indexing is preferable for very large sorted files where reducing index storage is important.

8. Indexed Sequential File Organization vs Direct File Organization

Question: Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Answer

Indexed Sequential File Organization (ISFO) maintains records in sequential order and uses an index for faster access. Direct file organization uses a direct addressing or hashing mechanism to locate records.

Feature	Indexed Sequential File	Direct File
Record organization	Sequentially ordered	Directly mapped to storage location/bucket
Exact search	Fast using index	Very fast using hash/direct address
Sequential access	Excellent	Less suitable
Range queries	Excellent	Poor compared with ordered structures
Insertions	May require overflow areas/reorganization	Generally fast
Deletion	Requires index/file maintenance	Usually simpler
Index requirement	Yes	Usually uses direct mapping/hash
Best suited for	Both sequential and indexed access	Exact-match access

Indexed Sequential File

Structure:

Index



Block 1 → Block 2 → Block 3 → Block 4

The index quickly locates the appropriate block, while the sequential arrangement allows efficient ordered processing.

Direct File

A hash function can be used:

$h(\text{Key}) = \text{Key} \text{ MOD } \text{Number_of_Buckets}$

Example:

$h(105) = 105 \text{ MOD } 10$
 $= 5$

The record is directed toward bucket 5.

Advantages of Indexed Sequential Organization

- Supports sequential processing.
- Supports direct access through the index.
- Excellent for range queries.
- Suitable for reporting applications.

Limitations

- Index requires additional storage.

- Insertions can create overflow areas.
- Maintaining sorted order can be costly.

Advantages of Direct Organization

- Very fast exact-match access.
- Good for high-frequency key-based searches.
- Does not require sequential scanning.

Limitations

- Poorer support for range queries.
- Collision/overflow management may be required.
- Sequential processing is less efficient.

Conclusion

Indexed sequential organization is better when both sequential and range access are required, while direct organization is better for high-speed exact-match retrieval.

9. Single-Level Indexing vs Multi-Level Indexing

Question: Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Answer

Single-level indexing uses one index structure between the search key and the data file. Multi-level indexing creates additional index levels to reduce the amount of index that must be searched.

Feature	Single-Level Index	Multi-Level Index
Number of levels	One	Multiple
Structure	Index → Data	Top Index → Lower Index → Data
Search	Efficient for smaller indexes	More efficient for very large indexes
Memory	Lower	Slightly higher due to additional levels
Scalability	Limited for huge datasets	Highly scalable
Disk accesses	Can increase as index grows	Reduced by hierarchical searching
Complexity	Simple	More complex

Single-Level Index

Index



Data File

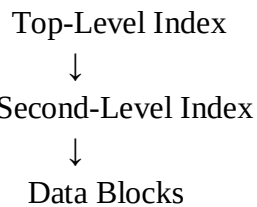
For example:

Index → [100, 200, 300, 400]



Data Blocks

Multi-Level Index



The upper-level index points to lower-level index blocks.

Analysis

Suppose an index itself becomes very large. Searching the entire index can require multiple disk accesses.

Multi-level indexing solves this by creating a hierarchy.

The concept is similar to a B-Tree/B+ Tree structure, where each level narrows the search space.

Conclusion

Single-level indexing is suitable for smaller indexes, while multi-level indexing is more suitable for large databases because it reduces the number of disk accesses required to locate records.

10. Hashing vs Indexing for Exact-Match and Range Queries

Question: Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Answer

Hashing and indexing are both used to improve database retrieval, but they are optimized for different types of queries.

Query Type	Hashing	B+ Tree Indexing
Exact-match search	Excellent	Excellent
Range search	Poor	Excellent
Ordered retrieval	Poor	Excellent
Average exact search	Approximately $O(1)$	Approximately $O(\log n)$
Range search	Requires multiple bucket checks/scanning	Approximately $O(\log n + k)$
Sequential traversal	Not naturally ordered	Efficient through leaf links
Collision handling	Required	Not applicable in the same way
Best application	Equality queries	Equality + range queries

Exact-Match Query

Example:

```
SELECT *
```

```
FROM Student
```

```
WHERE Student_ID = 105;
```

Hashing is highly effective.

A hash function can directly identify the bucket:

$h(105) \rightarrow \text{Bucket } 5$

Therefore, the average lookup can be approximately **$O(1)$** .

Range Query

Example:

SELECT *

FROM Student

WHERE Student_ID BETWEEN 100 AND 200;

Hashing is not suitable because hash functions destroy the natural ordering of keys.

A B+ Tree is much better because its keys are ordered:

$[100,110,120] \rightarrow [130,140,150] \rightarrow [160,170,180] \rightarrow \dots$

The database finds the first matching key and then follows the linked leaves.

Performance Evaluation

Hashing

Advantages:

- Very fast exact-match retrieval.
- Good for equality searches.
- Average lookup can approach $O(1)$.

Limitations:

- Poor for range queries.
- Collisions require additional handling.
- Does not preserve key ordering.

B+ Tree Index

Advantages:

- Efficient exact search.
- Excellent range queries.
- Maintains sorted keys.
- Supports ordered traversal.
- Suitable for large disk-based databases.

Limitations:

- Search is generally $O(\log n)$ rather than average $O(1)$.
- Index maintenance is required.
- Requires additional storage.

Final Evaluation

Exact Match:

Hashing $\rightarrow$ ☆☆☆☆☆

B+ Tree $\rightarrow$ ☆☆☆

Range Query:

Hashing $\rightarrow$ ☆☆

B+ Tree $\rightarrow$ ☆☆☆☆☆

Therefore:

Use hashing when the workload is dominated by exact-match queries.

Use B+ Tree indexing when the workload includes range queries, ordered retrieval, or both exact and range searches.

For a general-purpose database workload containing **both equality and range queries**, **B+ Tree indexing is the more versatile choice.**